

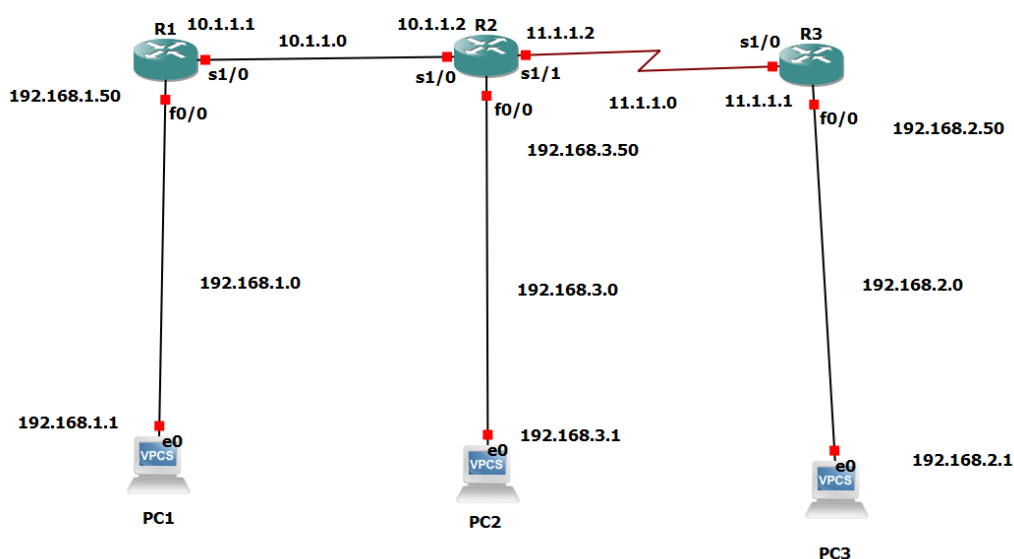
Practical No 2

AIM: Implement IPv4 ACLs

1. Standard

2. Extended

Step 1: Build the network as follow:



- Access-list (ACL) is a set of rules defined for controlling network traffic and reducing network attacks. ACLs are used to filter traffic based on the set of rules defined for the incoming or outgoing of the network.
- In a way, an ACL is like a guest list at an exclusive club.
- ACL is also called as packet filtering firewall
- There are two main different types of Access-list namely:
 1. **Standard Access-list** – These are the Access-list that are made using the source IP address only. These ACLs permit or deny the entire protocol suite. They don't distinguish between the IP traffic such as TCP, UDP, HTTPS, etc. These use range 1-99 or 1300-1999. Here we only give source IP Address.
 2. **Extended Access-list** – These are the ACL that uses source IP, Destination IP, source port, and Destination port. These types of ACL, we can also mention which IP traffic should be allowed or denied. These use range 100-199 and 2000-2699. Here we give both the source and destination IP Address.
- To enable ACL on our network we use Routing protocols because via connection is not reachable so we will be applying a RIP.
- After applying RIP all the Routers and PCs are able to communicate and ping each other.

Step 3: Configure IP to all the routers and PC's

R1:

```
en
conf t
int f0/0
ip add 192.168.1.50 255.255.255.0
no shut
ex
int s1/0
ip add 10.1.1.1 255.255.255.0
no shut
ex
do wr
```

```
R1#en
R1#conf t
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#int f0/0
R1(config-if)#ip add 192.168.1.50 255.255.255.0
R1(config-if)#no shut
R1(config-if)#ex
R1(config)#int s1/0
R1(config-if)#ip add 10.1.1.1 255.255.255.0
R1(config-if)#no shut
R1(config-if)#ex
R1(config)#do wr
Warning: Attempting to overwrite an NVRAM configuration previously written
by a different version of the system image.
Overwrite the previous NVRAM configuration?[confirm]y
Building configuration...
[OK]
```

R2:

```
en
conf t
int f0/0
ip add 192.168.3.50 255.255.255.0
no shut
ex
int s1/0
ip add 10.1.1.2 255.255.255.0
no shut
ex
```

```
int s1/1
ip add 11.1.1.2 255.255.255.0
no shut
ex
do wr
```

```
R2#en
R2#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#int f0/0
R2(config-if)#ip add 192.168.3.50 255.255.255.0
R2(config-if)#no shut
R2(config-if)#ex
R2(config)#int s1/0
R2(config-if)#ip add 10.1.1.2 255.255.255.0
R2(config-if)#no shut
R2(config-if)#ex
R2(config)#int s1/1
R2(config-if)#ip add 11.1.1.2 255.255.255.0
R2(config-if)#no shut
R2(config-if)#ex
R2(config)#do wr
Building configuration...
[OK]
R2(config)#
```

R3:

```
en
conf t
int f0/0
ip add 192.168.2.50 255.255.255.0
no shut
ex
int s1/0
ip add 11.1.1.1 255.255.255.0
no shut
ex
```

```
R3#en
R3#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R3(config)#int f0/0
R3(config-if)#ip add 192.168.2.50 255.255.255.0
R3(config-if)#no shut
R3(config-if)#ex
R3(config)#int s1/0
R3(config-if)#ip add 11.1.1.1 255.255.255.0
R3(config-if)#no shut
R3(config-if)#ex
R3(config)#
```

PC 1:

ip 192.168.1.1 255.255.255.0 192.168.1.50

sh ip

```
PC1 : 192.168.1.1 255.255.255.0 gateway 192.168.1.50

PC1> sh ip

NAME       : PC1[1]
IP/MASK     : 192.168.1.1/24
GATEWAY     : 192.168.1.50
DNS         :
MAC         : 00:50:79:66:68:00
LPORT      : 10024
RHOST:PORT  : 127.0.0.1:10025
MTU:        : 1500

PC1> 
```

PC 2:

ip 192.168.3.1 255.255.255.0 192.168.3.50

sh ip

```
PC2> ip 192.168.3.1 255.255.255.0 192.168.3.50
Checking for duplicate address...
PC1 : 192.168.3.1 255.255.255.0 gateway 192.168.3.50

PC2> sh ip

NAME          : PC2[1]
IP/MASK        : 192.168.3.1/24
GATEWAY        : 192.168.3.50
DNS            :
MAC            : 00:50:79:66:68:01
LPORT         : 10026
RHOST:PORT     : 127.0.0.1:10027
MTU:           : 1500

PC2> █
```

PC 3:

Ip 192.168.2.1 255.255.255.0 192.168.2.50

sh ip

```
PC3> ip 192.168.2.1 255.255.255.0 192.168.2.50
Checking for duplicate address...
PC1 : 192.168.2.1 255.255.255.0 gateway 192.168.2.50

PC3> sh ip

NAME          : PC3[1]
IP/MASK        : 192.168.2.1/24
GATEWAY        : 192.168.2.50
DNS            :
MAC            : 00:50:79:66:68:02
LPORT         : 10028
RHOST:PORT     : 127.0.0.1:10029
MTU:           : 1500

PC3> █
```

Step 4: After assigning the IP Address to all the router and PC try ping the neighbor PC or router. It should work successfully

R1:

ping 192.168.1.1

```
R1#ping 192.168.1.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.1.1, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 8/10/12 ms
R1#
```

R2:

ping 192.168.3.1

```
R2#ping 192.168.3.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.3.1, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 1/7/12 ms
R2#
```

R3:

ping 192.168.2.1

After that try ping to indirect connection. It won't be able to ping. Because we haven't configured any routing protocols for indirect connection to work.

So we will now configure RIP on all the router

Step 5: Follow below to configure RIP in routers.

R1:

```
config t
router rip
network 192.168.1.0
network 10.1.1.0
ex
do wr
```

```
R1(config)#router rip
R1(config-router)#network 192.168.1.0
R1(config-router)#network 10.1.1.0
R1(config-router)#ex
R1(config)#
```

R2:

```
config t
router rip
network 10.1.1.0
network 11.1.1.0
```

network 192.168.3.0

ex

```
R2#config t
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)#router rip
R2(config-router)#network 10.1.1.0
R2(config-router)#network 11.1.1.0
R2(config-router)#network 192.168.3.0
R2(config-router)#ex
R2(config)#
```

R3:

config t

router rip

network 11.1.1.0

network 192.168.2.0

ex

```
R3#config t
Enter configuration commands, one per line. End with CNTL/Z.
R3(config)#router rip
R3(config-router)#network 11.1.1.0
R3(config-router)#network 192.168.2.0
R3(config-router)#ex
R3(config)#
```

Once RIP is configured ping indirect connection. Now ping will work as we have applied RIP protocol.

Step 6: Now we will apply ACL.

As we have discussed there are 2 types of ACL. We will apply Standard ACL now.

R1:

Config t

access-list 10 deny host 192.168.2.1

exit

show access-list

config t

access-list 10 permit any

int s1/0

ip access-group 10 in

exit

do wr

```
R1(config)#access-list 10 deny host 192.168.2.1
R1(config)#exit
R1#
*Sep 18 19:08:15.703: %SYS-5-CONFIG_I: Configured from console by console
R1#show access-list
Standard IP access list 10
    10 deny    192.168.2.1
R1#config t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#access-list 10 permit any
R1(config)#int s1/0
R1(config-if)#ip access-group 10 in
R1(config-if)#exit
```

Here the scenario we are going to use is that whatever data that is being sent by PC 3 should not be received by PC 1, so based on the features of a standard ACL, we know that it needs to be placed near the destination.

The number of the standard ACL should be in the range of 1-99, in our case we have chosen 10.

So by configuring the access list to 'deny', we deny all communication from PC 3 to PC 1.

Here we can observe that when PC 3 tries to send data to PC 1, its access is denied.

PC3:

ping 192.168.1.1

```
PC3> ping 192.168.1.1
*10.1.1.1 icmp_seq=1 ttl=253 time=53.416 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.1 icmp_seq=2 ttl=253 time=46.518 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.1 icmp_seq=3 ttl=253 time=34.178 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.1 icmp_seq=4 ttl=253 time=41.668 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.1 icmp_seq=5 ttl=253 time=37.497 ms (ICMP type:3, code:13, Communication administratively prohibited)
```

R3:

conf t

access-list 40 deny host 192.168.3.1

do sh access-list

access-list 40 permit any

int s1/0

ip access-group 40 in

exit


```

R3(config)#no access-list 40
R3(config)#access-list 40 deny host 192.168.3.1
R3(config)#do sh access-list
Standard IP access list 40
    10 deny    192.168.3.1
R3(config)#access-list 40 permit any
R3(config)#int s1/0
R3(config-if)#ip access-group 40 in
R3(config-if)#exit
R3(config)#

```

Here R3 have block PC2. So PC3 wont be receiving data from PC2.

PC2:

Ping 192.168.2.1

```

PC2> ping 192.168.2.1
*11.1.1.1 icmp_seq=1 ttl=254 time=19.663 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=2 ttl=254 time=17.264 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=3 ttl=254 time=19.841 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=4 ttl=254 time=18.289 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=5 ttl=254 time=14.920 ms (ICMP type:3, code:13, Communication administratively prohibited)

```

R2:

```

conf t
access-list 10 deny 192.168.1.1
exit
sh access-list
conf t
access-list 10 permit any
int s1/0
ip access-group 10 in
exit
access-list 20 deny icmp host 192.168.1.1 host 192.168.2.1

```

```

R2#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#access-list 10 deny 192.168.1.1
R2(config)#exit
R2#
*Sep 18 19:20:55.107: %SYS-5-CONFIG_I: Configured from console by console
R2#sh access-list
Standard IP access list 10
    10 deny    192.168.1.1

```

```

R2#config t
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)#access-list 10 permit any
R2(config)#int s1/0
R2(config-if)#ip access-group 10 in
R2(config-if)#exit
R2(config)#access-list 20 deny icmp host 192.168.1.1 host 192.168.2.1
Translating "icmp"
          ^
% Invalid input detected at '^' marker.

```

Here R2 have block PC1. So if PC1 send data PC3 it will deny.

PC1> ping 192.168.3.1

```

PC1> ping 192.168.3.1
*10.1.1.2 icmp_seq=1 ttl=254 time=28.930 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.2 icmp_seq=2 ttl=254 time=31.267 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.2 icmp_seq=3 ttl=254 time=23.073 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.2 icmp_seq=4 ttl=254 time=27.738 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.1.1.2 icmp_seq=5 ttl=254 time=29.916 ms (ICMP type:3, code:13, Communication administratively prohibited)

```

Step 7: We will now apply Extended ACL

R3:

```
access-list 121 deny icmp host 192.168.3.1 host 192.168.2.1
```

```
do sh access-list 121
```

```
access-list 121 permit icmp any any
```

```
do sh access-list 121
```

```
int s1/0
```

```
ip access-group 121 out
```

```
do sh access-list 121
```

```
exit
```

```

R3(config)#access-list 121 deny icmp host 192.168.3.1 host 192.168.2.1
R3(config)#do sh access-list 121
Extended IP access list 121
  10 deny icmp host 192.168.3.1 host 192.168.2.1
R3(config)#access-list 121 permit icmp any any
R3(config)#do sh access-list 121
Extended IP access list 121
  10 deny icmp host 192.168.3.1 host 192.168.2.1
  20 permit icmp any any
R3(config)#int s1/0
R3(config-if)#ip access-group 121 out
R3(config-if)#do sh access-list 121
Extended IP access list 121
  10 deny icmp host 192.168.3.1 host 192.168.2.1
  20 permit icmp any any
R3(config-if)#exit

```

We will apply ACL on R3. It will deny PC2 to send data to PC3. Here we can observe that when PC 2 tries to send data to PC 3, its access is denied.

Here PC3 have accept R2 but deny PC2.

PC2> ping 192.168.2.1

```
PC2> ping 192.168.2.1
*11.1.1.1 icmp_seq=1 ttl=254 time=48.482 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=2 ttl=254 time=45.139 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=3 ttl=254 time=44.831 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=4 ttl=254 time=30.557 ms (ICMP type:3, code:13, Communication administratively prohibited)
*11.1.1.1 icmp_seq=5 ttl=254 time=46.231 ms (ICMP type:3, code:13, Communication administratively prohibited)
```

But if any other device ping PC 3 it will permit it

R2#ping 192.168.2.1

```
R2#ping 192.168.2.1
*Sep 18 19:36:18.851: %SYS-5-CONFIG_I: Configured from console by console
R2#ping 192.168.2.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.2.1, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 40/304/1088 ms
```

We can assume that we have successfully used an extended access list.